

慶應義塾大学 理工学部 情報工学科 自然言語処理 (2026/06/11)

09 - 言語モデルを事前学習する

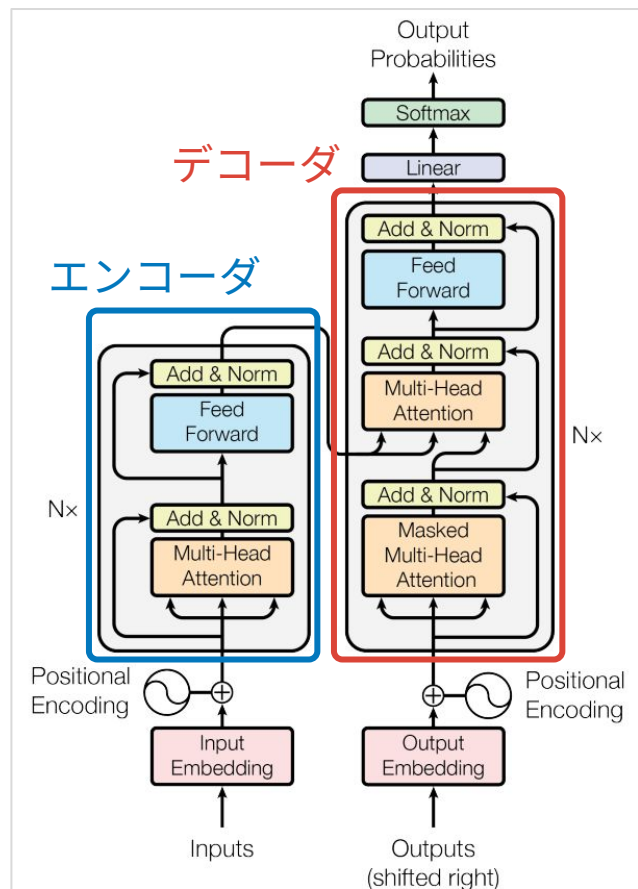
情報工学科 准教授 高道 慎之介

講義予定

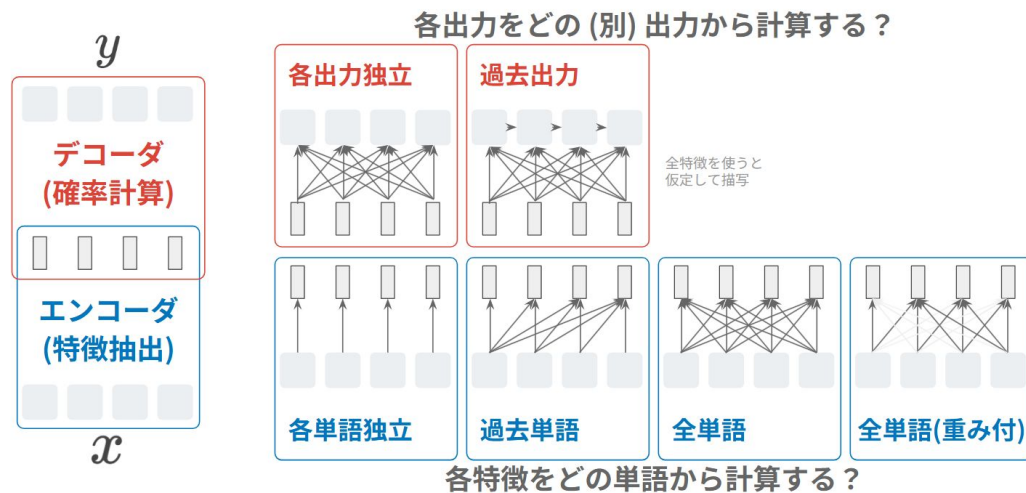
第XX回	日付	内容（順次変わっていくので予想）	
第01回	2026/04/09	イントロダクション	事前知識の確認
第02回	2026/04/16	自然言語処理のための機械学習に入門する	
第03回	2026/04/23	言語学に入門する	言語とは何だろう
第04回	2026/04/30	単語をベクトルで表現する	自然言語処理の基礎技術
第05回	2026/05/07	言語モデルで文章を統計的に扱う	
第06回	2026/05/14	系列にラベリングする	
第07回	2026/05/21	演習 1	これまでの復習
第08回	2026/05/28	Transformer を理解する	大規模言語モデル時代の技術
第09回	2026/06/11	言語モデルを事前学習する	
第10回	2026/06/18	言語モデルを転移学習する	
第11回	2026/06/25	言語を生成 (デコーディング) する	
第12回	2026/07/02	言語やラベルを評価する	おまけ
第13回	2026/07/09	自然言語処理から音声言語処理へ	
第14回	2026/07/16	演習 2	これまでの復習
期末試験	2026/07/XX	(日程は後日アナウンス)	

復習：Transformer

Transformer はエンコーダとデコーダから成る

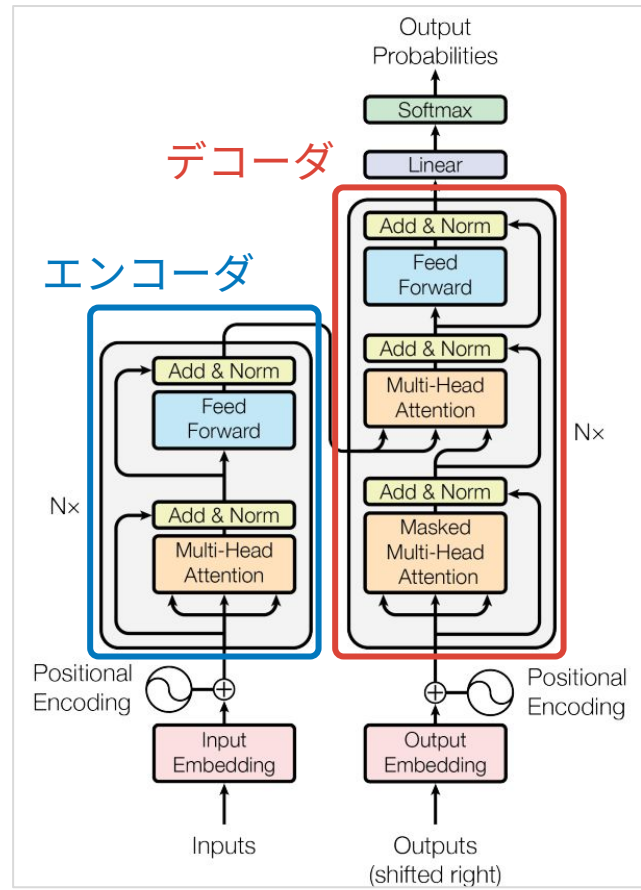
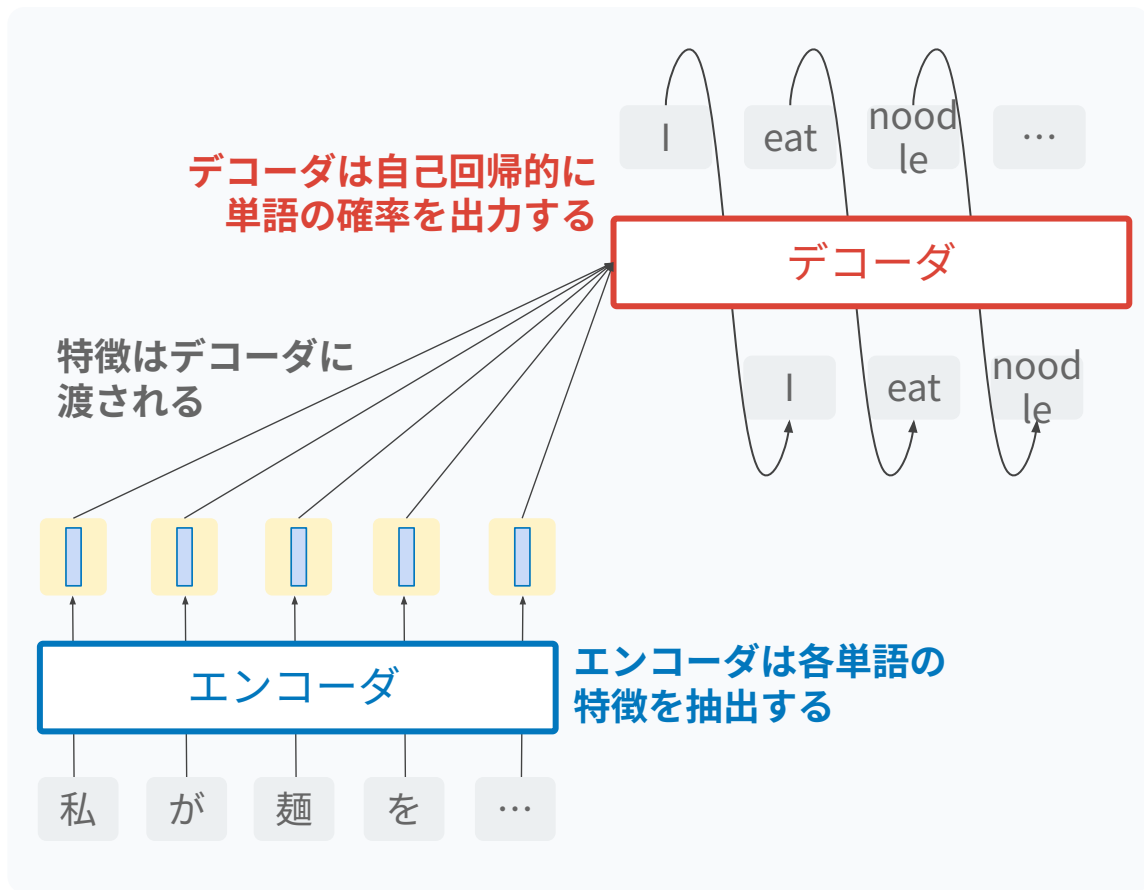


入力と出力が系列になった場合を考えよう



第 06 回スライドから引用

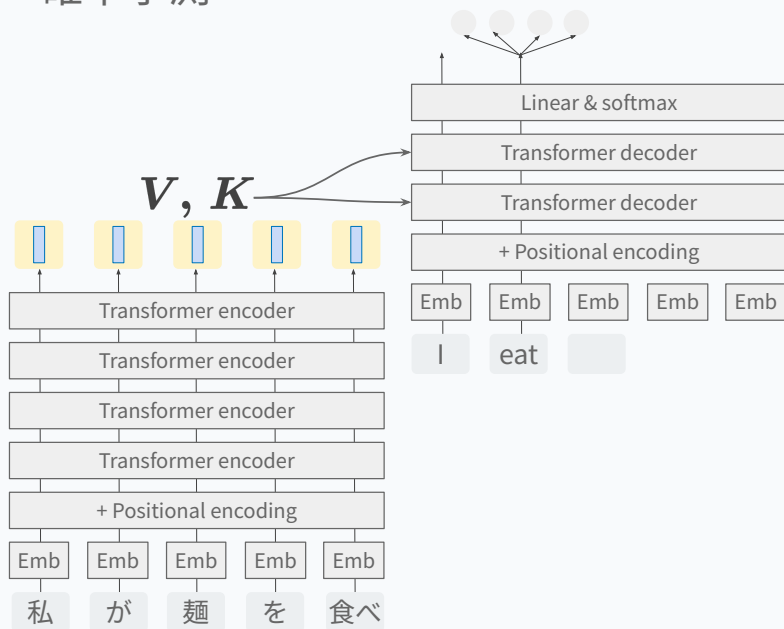
エンコーダとデコーダの役割



Transformer から GPT へ

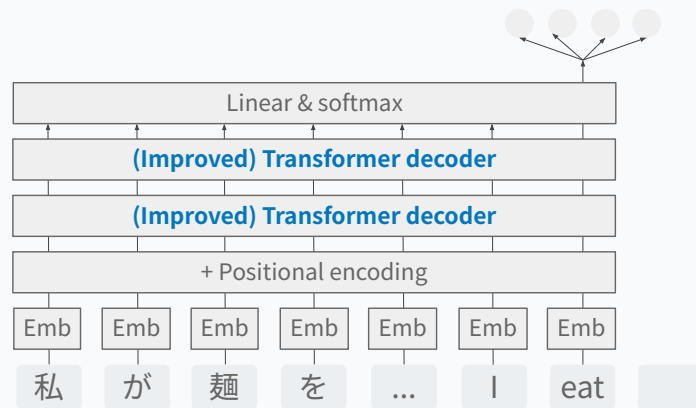
Transformer (encoder-decoder)

エンコーダで特徴抽出．デコーダで確率予測



GPT (decoder-only)

特徴抽出も確率予測もデコーダで．
基本は Transformer decoder と同じ
だが，いくつかの改良が入っている

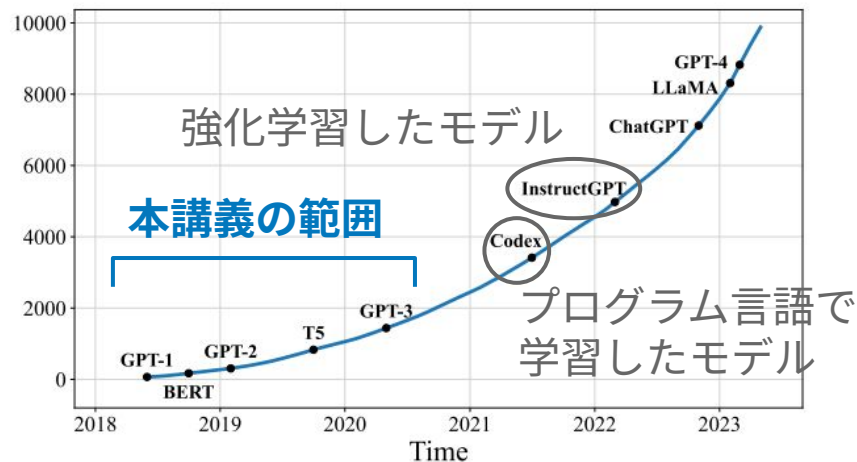


今日の目的：

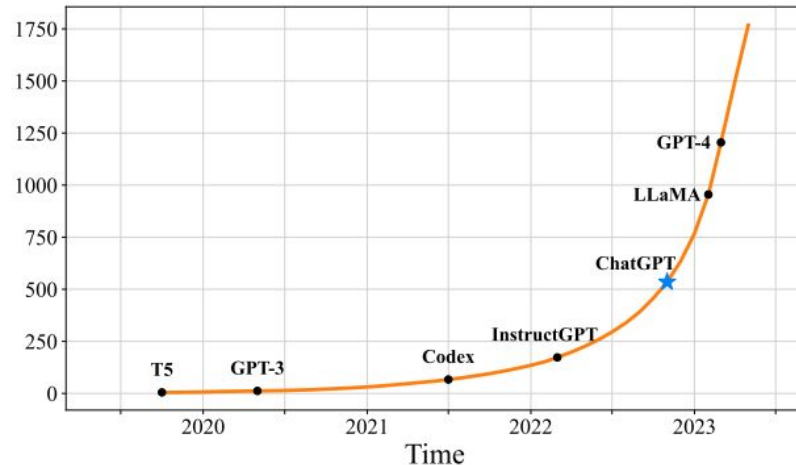
大規模言語モデルを事前学習することの意味を知る

大規模言語モデルの隆盛

大規模言語モデルの隆盛



(a) Query="Language Model"



(b) Query="Large Language Model"

Fig. 1: The trends of the cumulative numbers of arXiv papers that contain the keyphrases “*language model*” (since June 2018) and “*large language model*” (since October 2019), respectively. The statistics are calculated using exact match by querying the keyphrases in title or abstract by months. We set different x-axis ranges for the two keyphrases, because “*language models*” have been explored at an earlier time. We label the points corresponding to important landmarks in the research progress of LLMs. A sharp increase occurs after the release of ChatGPT: the average number of published arXiv papers that contain “*large language model*” in title or abstract goes from 0.40 per day to 8.58 per day (Figure 1(b)).

大規模言語モデルの隆盛

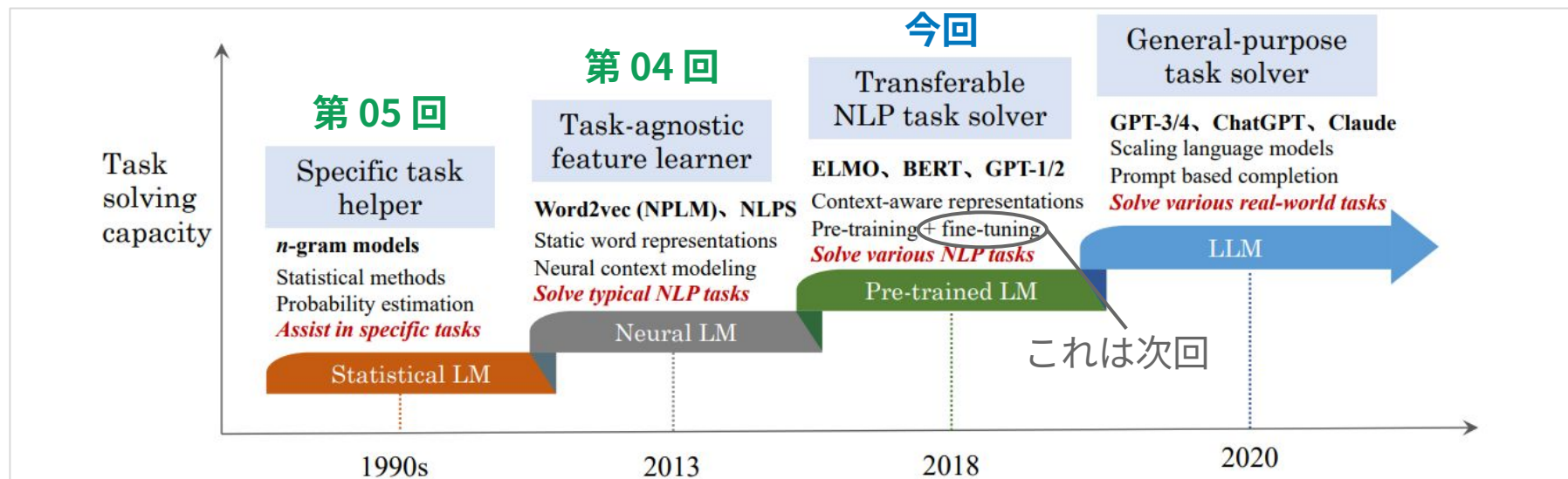
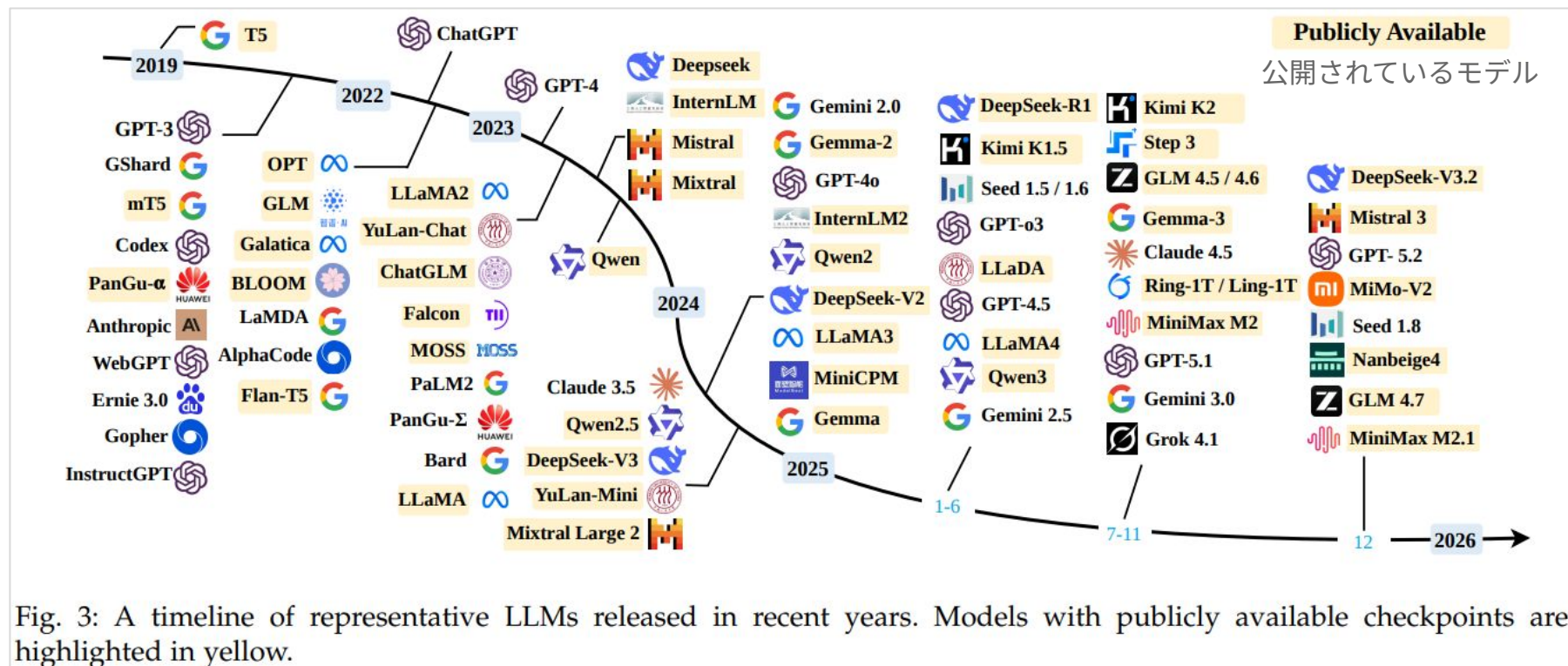


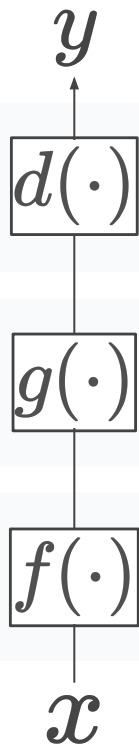
Fig. 2: An evolution process of the four generations of language models (LM) from the perspective of task solving capacity. Note that the time period for each stage may not be very accurate, and we set the time mainly according to the publish date of the most representative studies at each stage. For neural language models, we abbreviate the paper titles of two representative studies to name the two approaches: NPLM [1] (“A neural probabilistic language model”) and NLPS [2] (“Natural language processing (almost) from scratch”). Due to the space limitation, we don’t list all representative studies in this figure.

大規模言語モデルの隆盛



大規模言語モデルと“事前”学習

復習：x から y の予測



決定 (decision)

- 出力候補のうちから 1 つを決定すること

スコア計算 (score calculation)

- 特徴に基づいて，出力候補とそのスコアを計算すること

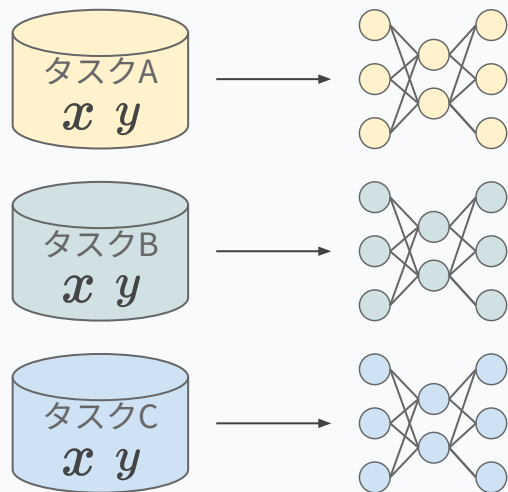
特徴抽出 (feature extraction)

- 意味のある特徴を入力テキストから抽出すること

伝統的な機械学習 vs. 最近の機械学習

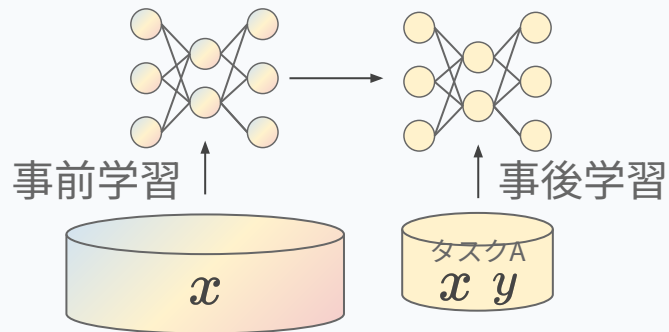
伝統的な機械学習

1. データ (x, y) のペアを大量に用意
2. モデル $f()$, $g()$ を乱数で初期化
3. (x, y) を使って $f()$, $g()$ を学習
 - a. supervised learning



最近の機械学習

1. x のみ (or y のみ) を大量に用意
2. モデル $f()$, $g()$ を乱数で初期化
3. x (or y) を使って $f()$, $g()$ を学習
4. データ (x, y) のペアを少量だけ用意
5. (x, y) を使って $f()$, $g()$ を微調整
 - a. “supervised fine-tuning (SFT)”



“事前”学習 (pre-training) の意味

大量のラベル無しデータ (テキストのみ) を学習に利用できる

ニューラルネットワークのスケラビリティの恩恵を受けられる

少量のラベル付きデータを準備するだけで良い

データ収集と学習に関するコストを抑えられる

広範な NLP タスクに転用できる特徴抽出・スコア計算を学習可能

ラベル無しデータだけで事前学習して「良い」パラメータを事前に推定可能

どれくらい大規模なデータ？

最近のモデルの事前学習データ

	トークン数 [兆]
gpt-oss120B	数兆
Llama3.1	15~
DeepSeek-v3	14.8
Qwen3	36
GPT-3	0.5
OLMo 2	3.9

LLM 大規模言語モデル講座 講義資料 © 2025 (CC BY-NC-ND 4.0)

LLM .jp v4 モデルの事前学習データ

サブワードが平均 2文字/トークンなら、
24兆文字相当 10万文字/冊とするなら2.4億冊相当？

全体の学習コーパス量 (合計 11.74T tokens)

- Pre-training Phase1 6.29T tokens
(4096x512x3Msteps / 4096x1024x1.5M steps)
- Pre-training Phase2 4.19T tokens
(4096 x 2048x500K steps)
- Mid-training Phase1 0.84T tokens
(4096 x 2048 x 100K steps)
- Mid-training Phase2 0.21T tokens
(16K x 512 x 25K steps)
- Mid-training Phase3 0.21T tokens
(64K x 128 x 25K steps)

<https://llm-jp.nii.ac.jp/resources/> (2026-05-19)

スケーリング則

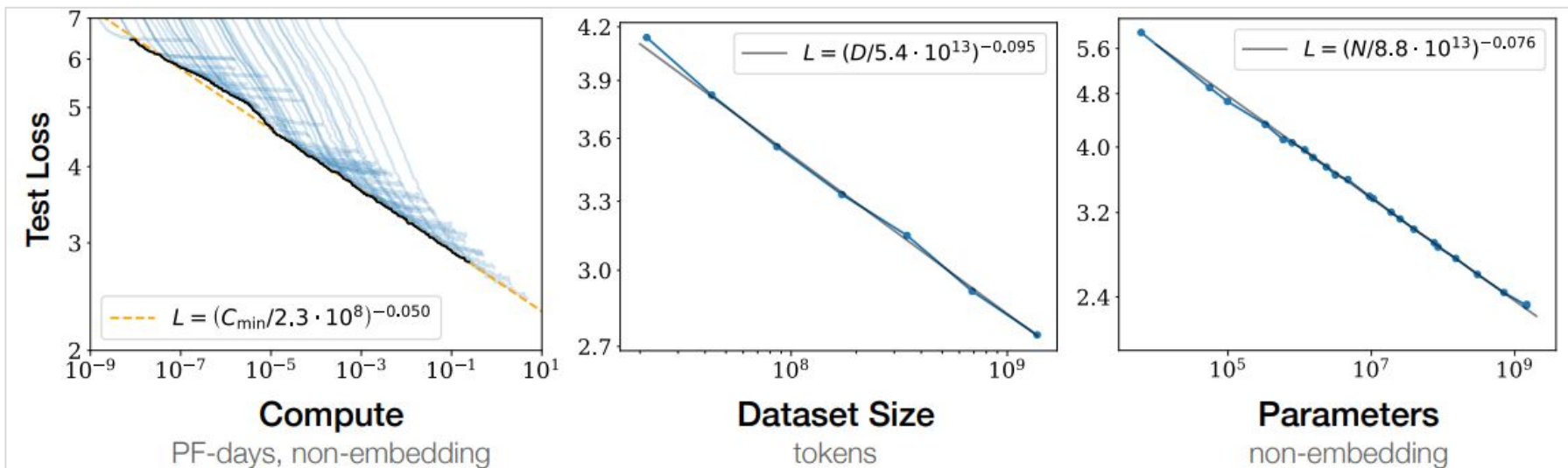


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

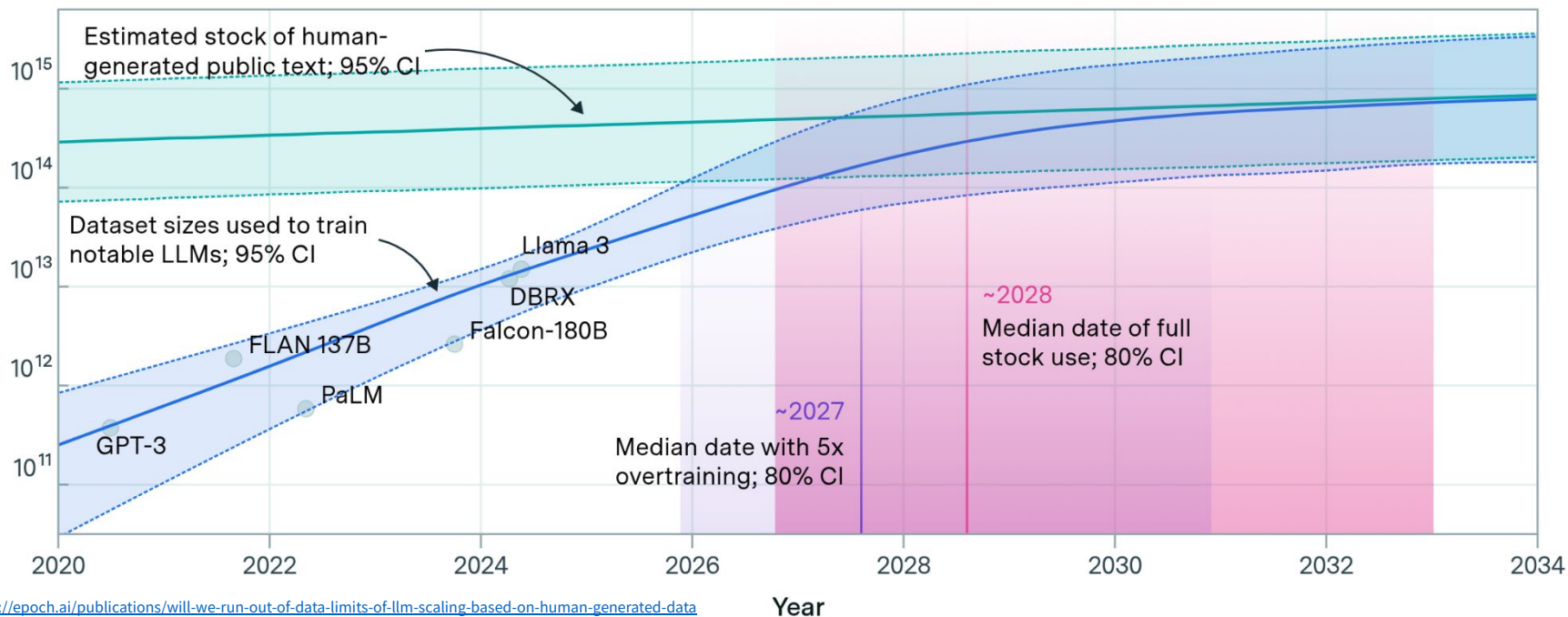
学習回数・事前学習データサイズ・モデルパラメータ数の対数に性能が比例する

余談：最近ではデータの枯渇も問題

Projections of the stock of public text and data usage



Effective stock (number of tokens)



LLM の事前学習に対するアプローチ

Option 1 商用LLMのAPIを使う	Option 2 既存のオープンソースLLMを利用する	Option 3 自らもしくは外部支援を受けてLLMを トレーニングする
長 所		
● 必要なLLMトレーニングの技術力が最も少ない。 ● 主なコストは推論時に発生するため、トレーニングや探索のコストを最小限に抑えることができる。 ● 最もデータ要求の少ない選択肢。モデルが推論を行うために必要なのは、わずかな例だけ(あるいは例なし)でよい。 ● 市場で最もパフォーマンスの高いLLMを活用し、優れた体験を構築できる。 ● LLMモデルを使ったアプリの市場投入までの時間を短縮し、プロジェクトのリスクを軽減できる。	● ①と比較すると、LLMサービスプロバイダーの将来の方向性への依存度が低い ため、ロードマップや後方互換性をコントロールしやすい。 ● ③と比較すると、LLMをゼロから構築しないため、データ、トレーニング時間、トレーニング予算が少なく済み、Time-to-Valueが大幅に短縮される。 ● LLMが膨大なインターネットデータから学んだことを活用し、推論時に知財にお金を払うことなく、構築できる。	● ①、②に比べ、LLMのパフォーマンスと将来の方向性を最もコントロールできるため、技術革新や下流タスクへのカスタマイズの自由度が高い。 ● モデルの品質、バイアス、有害性の問題に直接影響する学習用トレーニングデータセットを完全にコントロールすることができる。これに対し、①や②では、これらの問題をコントロールすることは不可能。 ● LLMをトレーニングすることで、強固な優位性を築くことができる。水平方向のユースケースや垂直方向に合わせたLLMの優れたパフォーマンスにより、特にLLMの展開を可能にするデータ/フィードバックループを作成した場合、持続的な優位性を構築することができる。

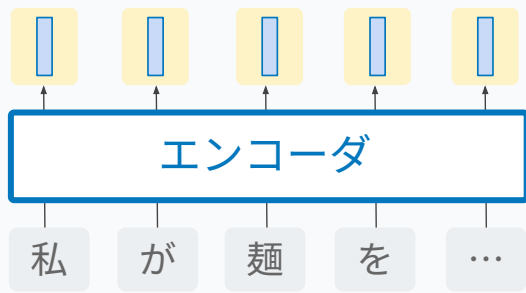
LLM の事前学習に対するアプローチ

Option 1 商用LLMのAPIを使う	Option 2 既存のオープンソースLLMを利用する	Option 3 自らもしくは外部支援を受けてLLMを トレーニングする
短 所		
● 商用LLMサービスは、微調整や推論タスクが大量に発生すると高価になることがある。これは、LLMの総所有コスト(TCO)を各推論に償却したものです。 ● 多くの業界やユースケースでは、コンプライアンス上、機密データやPIIデータの閲覧を許可できないため、商用LLMサービスの利用を禁じている(例えば、ヘルスケアユースケース)。 ● 外部アプリを構築する場合、外部のLLMサービス技術への依存度が高い場合は、他の強みを探し、事業のリスクを軽減する必要がある。 ● 下流の柔軟性が低い: エッジ推論をサポートしていない、モデルをカスタマイズする機能が限られている(微調整をするとコストがかかる)、モデルを継続的に改善する機能が限られている。	● 自分で作る時ほどではないが、オープンソースのLLMをトレーニングし、微調整し、ホストするためには、多くのドメイン専門家のスキルが必要である。LLMの再現性は依然として重要な問題であるため、必要な作業量と時間は過小評価できない。 ● より垂直な技術スタックのため、下流のアプリを構築している場合、市場投入までの時間が遅く、アジャイル性が低い。 ● オープンソースのモデルは、通常、商用モデルと比較して数ヶ月から数年単位で性能が遅れている。競合他社が商用モデルを利用している場合、彼らはLLM技術で優位に立っている可能性があるため、他の競争優位性を見つける必要がある。	● 非常に高価で、リスクも高い。NLP/ML、専門知識、ソフトウェア、ハードウェアの専門知識など、領域横断的な知識が必要。うまくやらないと、最適でないモデルで何千ドル、何百万ドルも費やしてしまったという事態になりかねない。特にトレーニングの後半でのミスは、修正・解消するのが難しい。 ● ②より効率が悪い。②は、インターネット上の全データから学習した既存のLLMを活用し、確度の高い初期モデルを提供することができる。③では、ゼロから始めることになり、モデルが汎化性能を獲得するためには、高品質で多様なデータセットが膨大に必要。

大規模言語モデルアーキテクチャの型

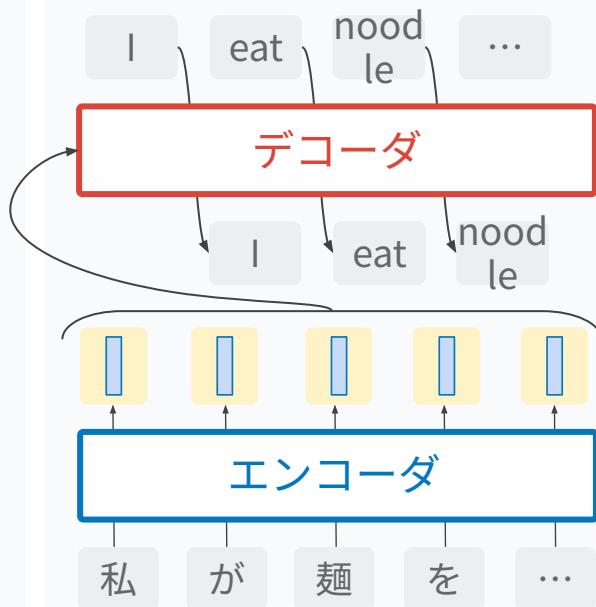
Encoder-only 型

BERT, RoBERTa, etc.



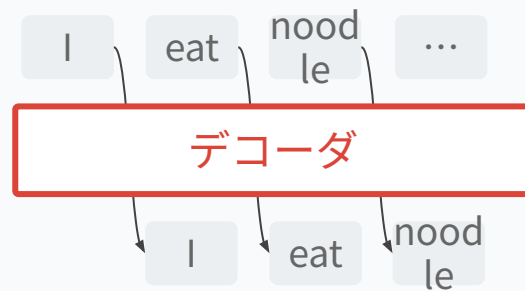
Encoder-decoder 型

BART, T5, etc.



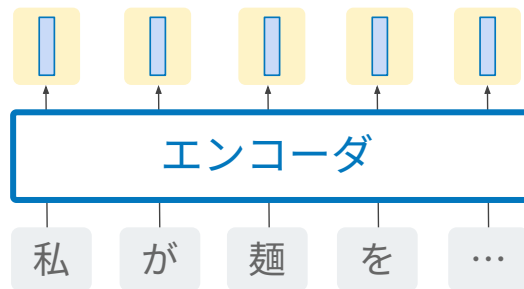
Decoder-only 型

GPT, LLaMA, etc.

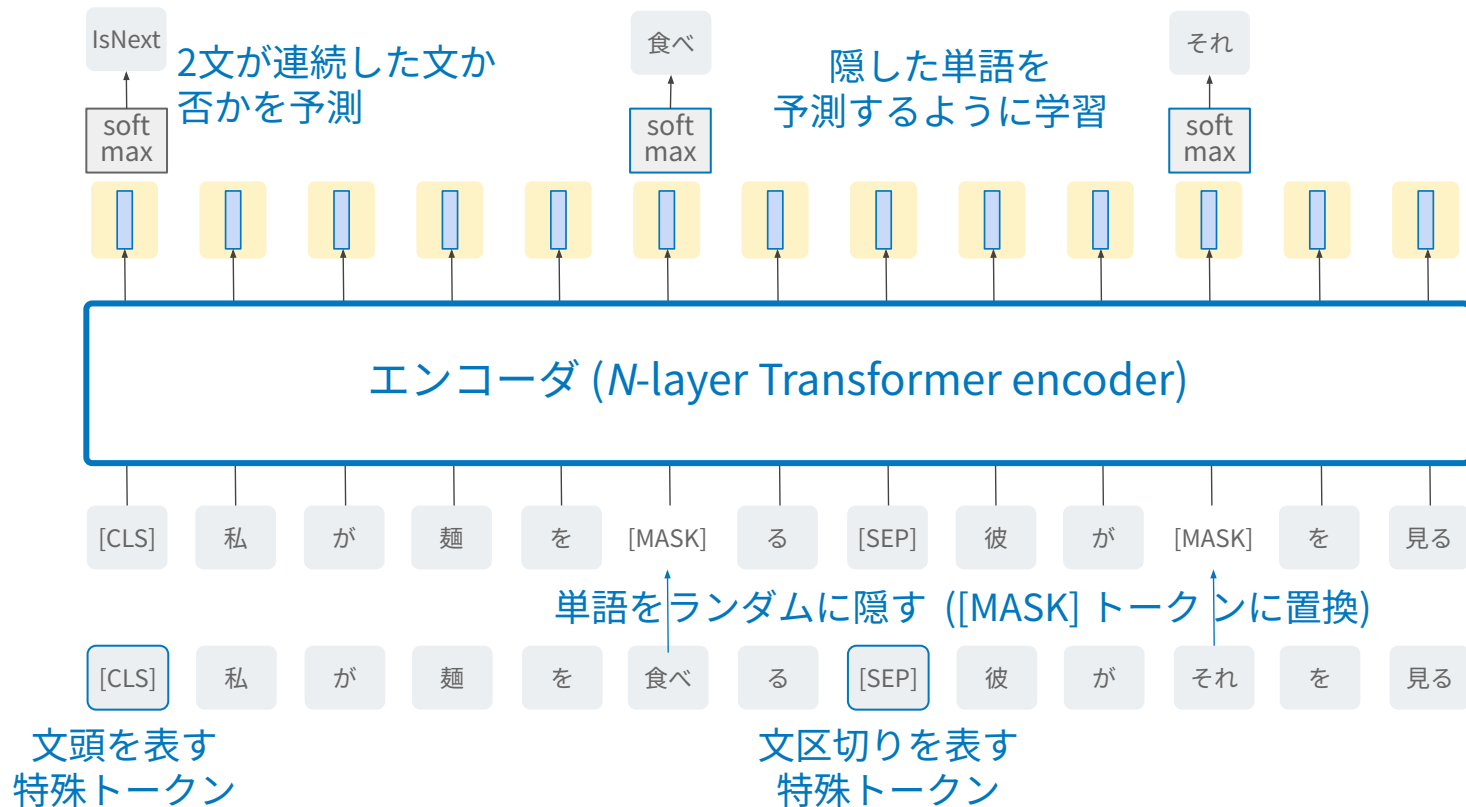


それぞれの型をどうやって事前学習するのかを知ろう

Encoder-only 型の事前学習



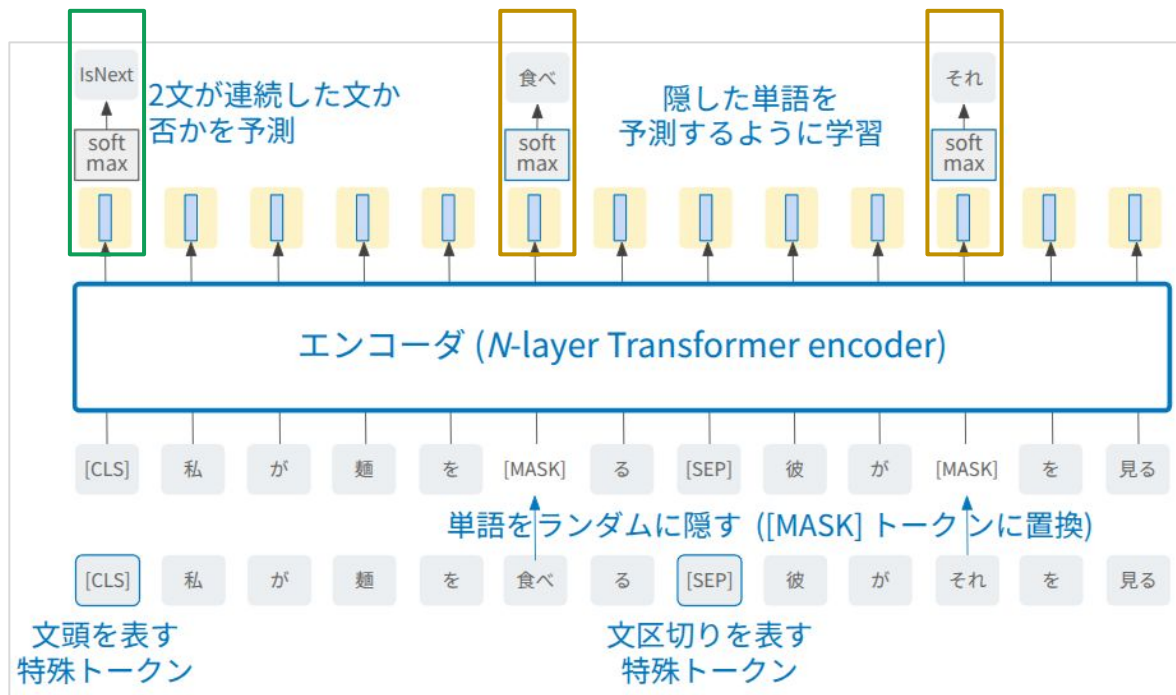
BERT：マスク言語モデルに基づく事前学習



BERTの気持ち

2文が関係していることを学習。
例えば質問文に対する回答文
→ 文レベルの埋め込みを学習

周りの単語から隠れた単語を予測するように学習
→ 分布仮説に基づいた「文脈による表現学習」
(得られたベクトルは“文脈”単語埋め込みと呼ばれる)

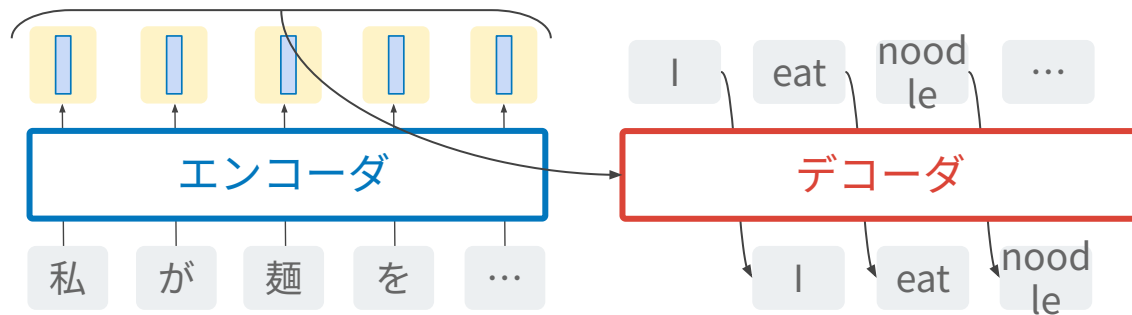


RoBERTa : BERT の改善

	MNLI	QNLI	QQP	RTE	SST	MRPC	CoLA	STS	WNLI	Avg
<i>Single-task single models on dev</i>										
BERT _{LARGE}	86.6/-	92.3	91.3	70.4	93.2	88.0	60.6	90.0	-	-
XLNet _{LARGE}	89.8/-	93.9	91.8	83.8	95.6	89.2	63.6	91.8	-	-
RoBERTa	90.2/90.2	94.7	92.2	86.6	96.4	90.9	68.0	92.4	91.3	-

- BERT と同じアーキテクチャ
- Next sentence prediction を排除 (masked LM のみ)
- マスクする単語を動的に決定
- データサイズを 10 倍以上に

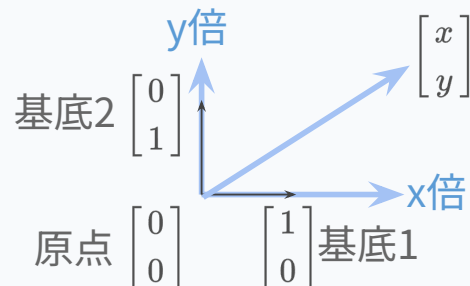
Encoder-decoder 型の事前学習



復習：線形代数・線形変換・アフィン変換

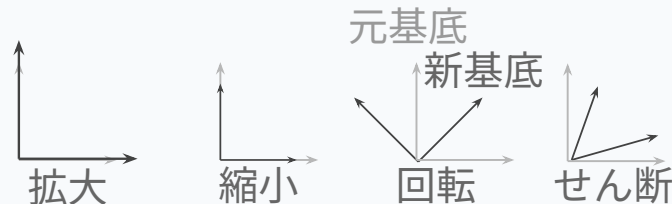
ベクトルは「基底ベクトルを何倍するか」の集まり

$$\begin{bmatrix} x \\ y \end{bmatrix} = x \begin{bmatrix} 1 \\ 0 \end{bmatrix} + y \begin{bmatrix} 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$



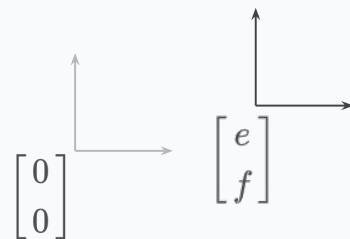
線形変換 $y = Ax$ は基底ベクトルの入れ替え．行列は新たな基底ベクトルの集まり

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix} = x \begin{bmatrix} a \\ c \end{bmatrix} + y \begin{bmatrix} b \\ d \end{bmatrix}$$



アフィン変換 $y = Ax + b$ は基底ベクトルの入れ替え&シフト

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} e \\ f \end{bmatrix} = \begin{bmatrix} ax + by + e \\ cx + dy + f \end{bmatrix} = x \begin{bmatrix} a \\ c \end{bmatrix} + y \begin{bmatrix} b \\ d \end{bmatrix} + \begin{bmatrix} e \\ f \end{bmatrix}$$



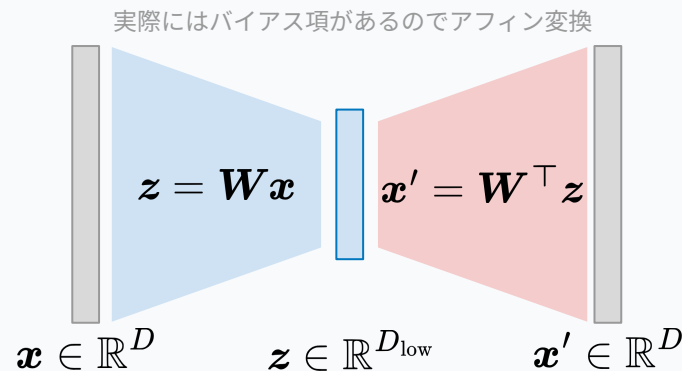
主成分分析と自己符号化器

主成分分析 (PCA)

高次元のデータを低次元ベクトルで表現。

その変換は線形であることを仮定。

変換行列は、データの共分散行列の固有ベクトルから成る。

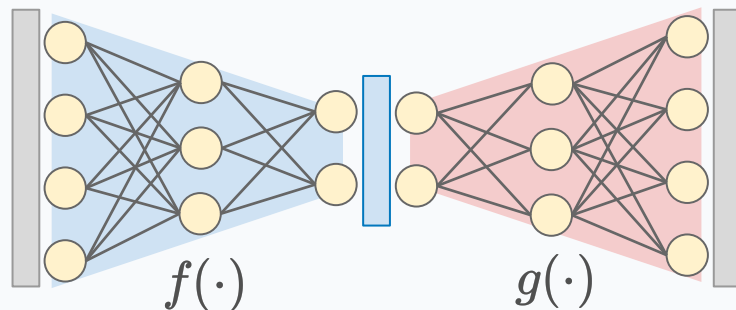


自己符号化器 (auto-encoder)

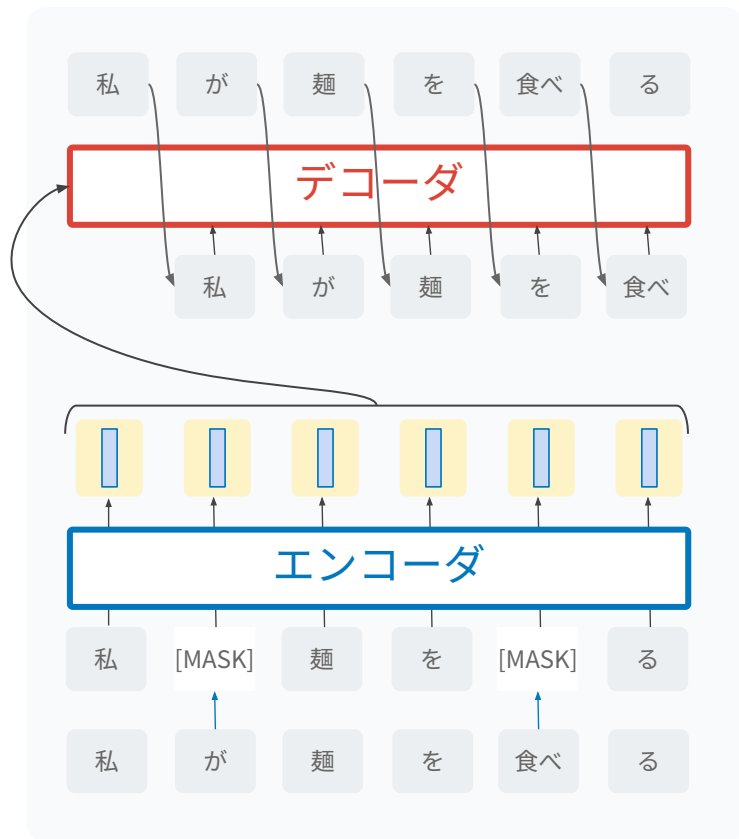
高次元のデータを低次元ベクトルで表現。

その変換は一般的に非線形。

変換パラメータは、データを再構成するように学習



BART: ディノイズングオートエンコーダに基づく事前学習



Encoder-only 型と比べて

- 特徴量抽出 (encoder) だけでなくテキスト生成 (decoder) も事前学習可能

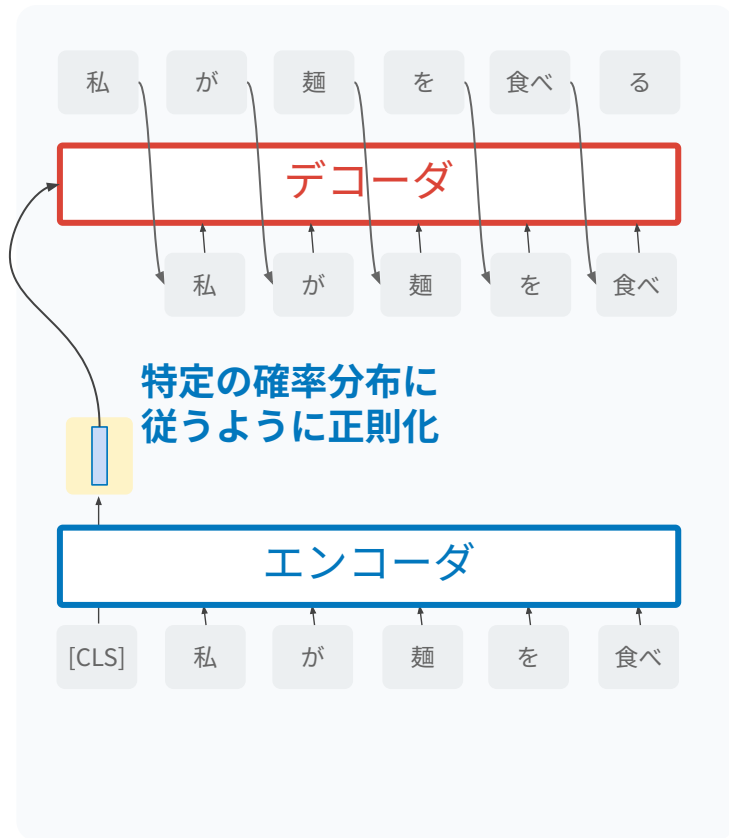
Decoder-only 型と比べて

- 全単語の関係から特徴を抽出可能 (decoder-only は過去の単語のみ)

Denoising の意味

- 壊れた文から元の文を復元するように学習することは、抽象的要約と同じ効果

OPTIMUS：変分オートエンコーダに基づく事前学習



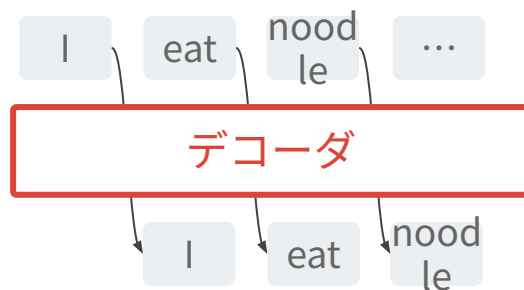
変分オートエンコーダ (VAE)

- 潜在変数が特定の確率分布に従うことを仮定するオートエンコーダ
- 一般的に「似ているデータ」は「似ている潜在変数」になりやすい

文レベルのベクトルを正規化

- アナロジーに基づく制御が可能

Decoder-only 型の事前学習



Decoder-only 型の事前学習



次トークン予測 (next token prediction)で学習

- t 番目の単語を，それ以前の単語から予測

Encoder-decoder 型と比べて

- KV キャッシュによる計算高速化が可能 (課題)
- 推論コストが小さい
 - encoder は self-attention で双方向計算を行うため重い
 - decoder は過去トークンのみ
- 長文に対する推論コストが小さい

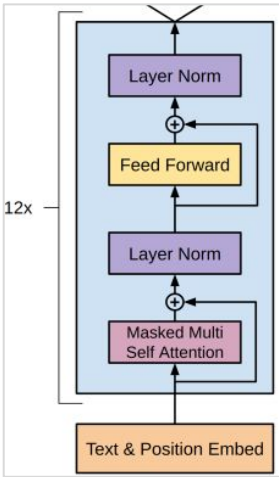
昨今の大規模言語モデルの殆どは decoder-only 型

TABLE 5: Model cards of several selected LLMs with public configuration details. Here, PE denotes position embedding, #L denotes the number of layers, #H denotes the number of attention heads, d_{model} denotes the size of hidden states, and MCL denotes the maximum context length during training.

Model	Category	Size	Normalization	PE	Activation	Bias	#L	#H	d_{model}	MCL
GPT3 [55]	Causal decoder	175B	Pre LayerNorm	Learned	GeLU	✓	96	96	12288	2048
PanGU- α [84]	Causal decoder	207B	Pre LayerNorm	Learned	GeLU	✓	64	128	16384	1024
OPT [90]	Causal decoder	175B	Pre LayerNorm	Learned	ReLU	✓	96	96	12288	2048
PaLM [56]	Causal decoder	540B	Pre LayerNorm	RoPE	SwiGLU	×	118	48	18432	2048
BLOOM [78]	Causal decoder	176B	Pre LayerNorm	ALiBi	GeLU	✓	70	112	14336	2048
MT-NLG [113]	Causal decoder	530B	-	-	-	-	105	128	20480	2048
Gopher [64]	Causal decoder	280B	Pre RMSNorm	Relative	-	-	80	128	16384	2048
Chinchilla [34]	Causal decoder	70B	Pre RMSNorm	Relative	-	-	80	64	8192	-
Galactica [35]	Causal decoder	120B	Pre LayerNorm	Learned	GeLU	×	96	80	10240	2048
LaMDA [68]	Causal decoder	137B	-	Relative	GeGLU	-	64	128	8192	-
Jurassic-1 [107]	Causal decoder	178B	Pre LayerNorm	Learned	GeLU	✓	76	96	13824	2048
LLaMA [57]	Causal decoder	65B	Pre RMSNorm	RoPE	SwiGLU	×	80	64	8192	2048
LLaMA 2 [99]	Causal decoder	70B	Pre RMSNorm	RoPE	SwiGLU	×	80	64	8192	4096
Falcon [171]	Causal decoder	40B	Pre LayerNorm	RoPE	GeLU	×	60	64	8192	2048
GLM-130B [93]	Prefix decoder	130B	Post DeepNorm	RoPE	GeGLU	✓	70	96	12288	2048
T5 [82]	Encoder-decoder	11B	Pre RMSNorm	Relative	ReLU	×	24	128	1024	512

Causal decoder が decoder-only モデル

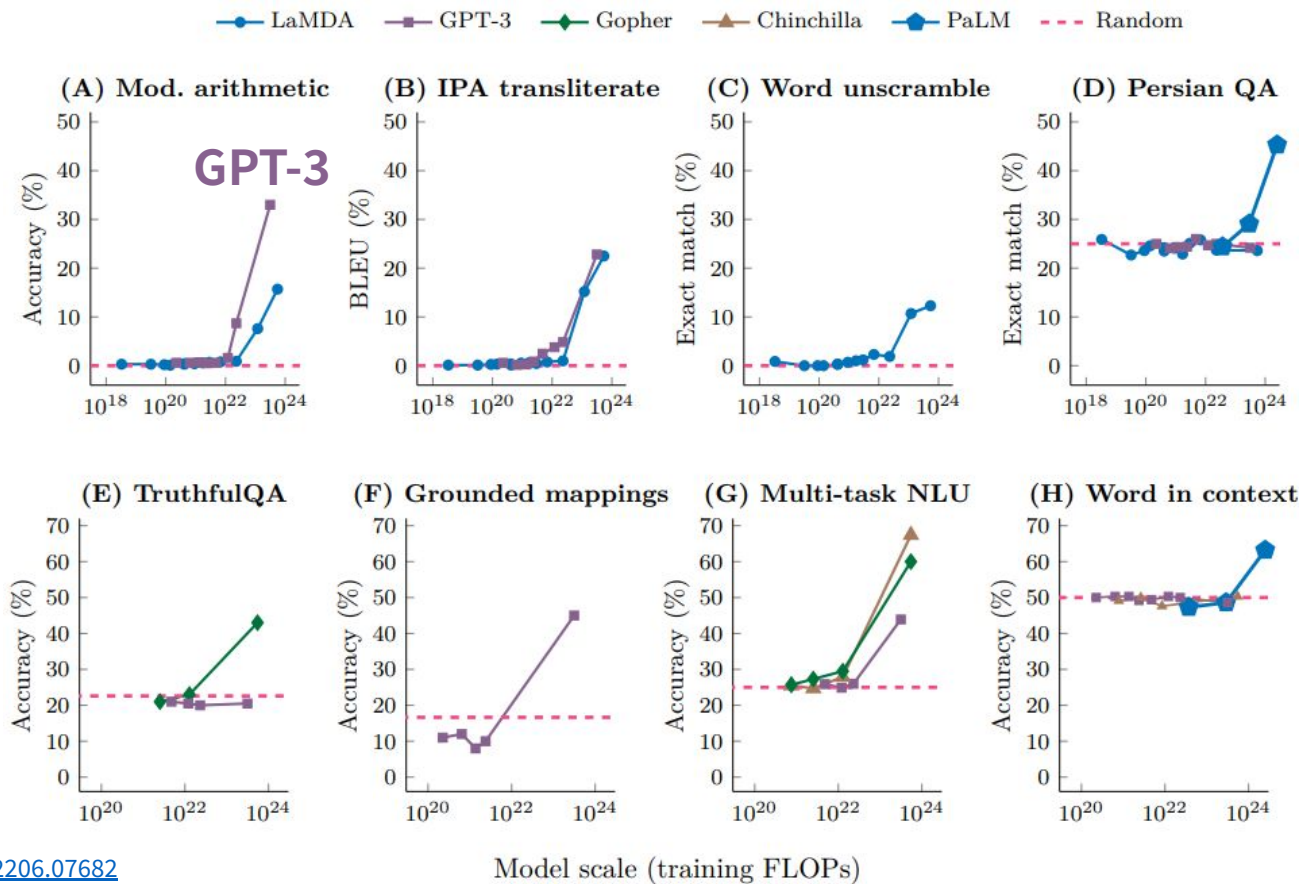
GPT 1 -> GPT 2 -> GPT 3

	GPT-1 (2018)	GPT-2 (2019)	GPT-3 (2020)
モデル構成	 The diagram illustrates the GPT-1 architecture. It starts with a 'Text & Position Embed' block at the bottom. This is followed by a stack of 12 identical layers, indicated by a bracket and '12x'. Each layer contains a 'Masked Multi Self Attention' block, a residual connection (indicated by a circle with a plus sign), a 'Layer Norm' block, a 'Feed Forward' block, another residual connection, and a final 'Layer Norm' block. The output of the stack is shown at the top.	• Layer norm の位置を各 sub ブロックの前に移動 • 乱数初期化の方法を変更	• 基本的にGPT2と同じ • 全行列の注意機構と通常 Transformer と, 一部要素のみを注意する <u>Sparse Transformer</u> の併用
モデルサイズ	1.2億 (0.12B)	15億 (1.5B)	1750億 (175B)

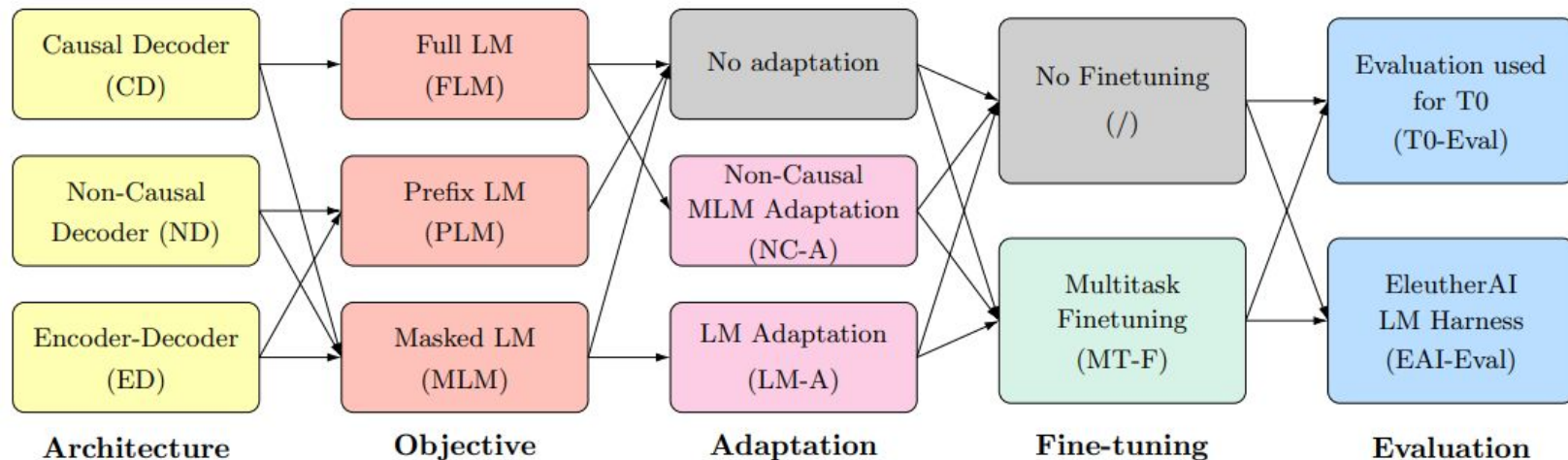
10 倍

100 倍

あるモデルサイズを超えると、スケーリング則を超えて急に性能が向上する



本当に decoder-only モデルが良いのか？



Decoder-only モデル

文全ての next token prediction

Finding 1. Causal decoder-only models pretrained with a **full language modeling** objective achieve best zero-shot generalization when evaluated immediately after **unsupervised pre-training**, in line with current common practices for large language models.

事前学習 (ラベルを使わないので“unsupervised”と書いてある)

事前学習データを作る

学習データの大半はウェブデータ

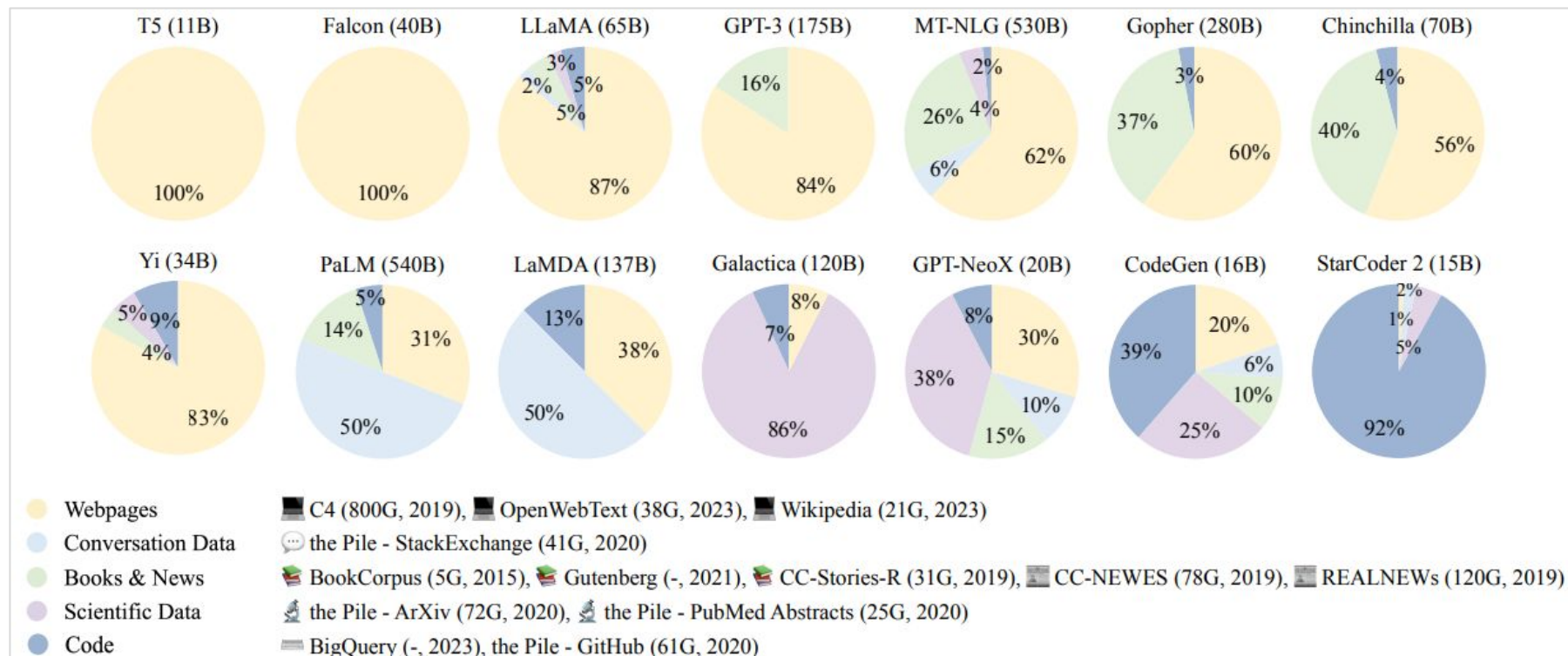


Fig. 6: Ratios of various data sources in the pre-training data for existing LLMs.

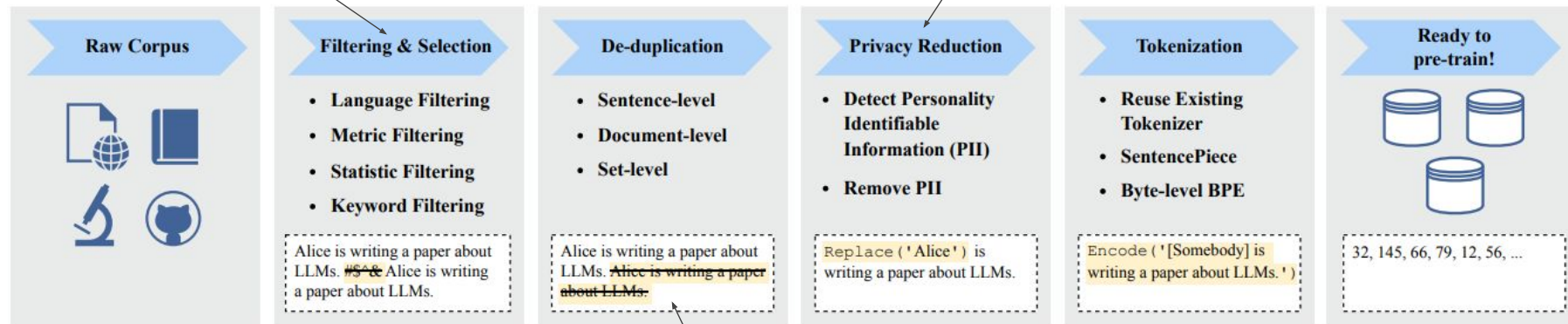
データに対する前処理

低品質データの排除

- 必要な言語のテキスト
- ヒューリスティックな基準
- 不適切キーワード

個人特定情報の削除

- 特定情報の検出
- 特定情報の削除



日本語 LLM のためのデータ前処理例
https://www.anlp.jp/proceedings/annual_meeting/2024/pdf_dir/P3-25.pdf

<https://arxiv.org/pdf/2303.18223>

重複データの排除

- 文・文章単位の重複の削除 (例えばウェブデータの引用によるダブルカウント)

まとめ

まとめ

- 大規模言語モデルの隆盛
- 事前学習の意味
- {Encoder-only, encoder-decoder, decoder-only} 型の事前学習
- 事前学習データを作る

Decoder-only 型の事前学習



次トークン予測 (next token prediction) で学習

- t 番目の単語を，それ以前の単語から予測

Encoder-decoder 型と比べて

- KV キャッシュによる計算高速化が可能 (課題)
- 推論コストが小さい
 - encoder は self-attention で双方向計算を行うため重い
 - decoder は過去トークンのみ
- 長文に対する推論コストが小さい

KVキャッシュとは何か，なぜKVキャッシュでdecoder-only 型の推論が高速になるかを，自分が理解できる粒度で述べよ．